Lab 04 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, an example you can check at the terminal.

Question 1 — `COPY ../common/…`

Answer. The build only has access to the **context** — the folder passed as argument (`.`, so `~/projects/api/`). `../common` is outside: Buildah brings the path back inside the context (`possible escaping context directory`), finds nothing there, and fails. `-f` only designates the Dockerfile, not the perimeter; an absolute path is also brought back into the context; `sudo` changes nothing about a perimeter problem, not a permissions one. Solution: build from the parent folder (`podman build -f api/Dockerfile -t api:1.0 ~/projects`) with `COPY api/… common/…` paths, or copy `config.yml` into the project before the build — or better, do not ship it at all and inject it at run time (lab 08).

Why. The context is a security and reproducibility boundary: a Dockerfile can only depend on what it is explicitly given. With Docker it is physical (the context is archived and sent to the daemon); with Podman it is a rule enforced by Buildah — same result.

Nuance. Podman accepts several named contexts: `podman build --build-context common=../common .` then `COPY --from=common config.yml /app/`. That is the clean solution when a shared file really has to enter several images.

Example.

```
podman build -f Dockerfile.out-of-context -t attempt .
# Error: building at STEP "COPY ../common/secret.txt /": … possible escaping context directory
error
```

Question 2 — `transferring context`, then nothing

Answer. Under Docker, the client packages the **whole** folder (1.1 GB) and sends it to the daemon before the first instruction: that is the `transferring context`. Under Podman, Buildah reads the folder in place, without archive or transfer: the slowness disappears. But the second risk is intact: a `COPY . .` ships `node_modules` and `.git` **into the image** — 1.1 GB of useless content, plus the full Git history (and its possible secrets) offered to anyone who downloads the image. `.dockerignore` therefore remains mandatory; it no longer serves speed, it serves content.

Why. The context has two roles: what is *sent* (Docker only) and what is *copyable*. Podman removes the first cost, not the second.

Nuance. `.git` inside an image is a frequent and serious leak: the history often contains credentials removed "since". And without `.dockerignore`, a file modified in `node_modules` also invalidates the `COPY` cache.

Example.

```
podman build -q -f Dockerfile.all -t t .           # 218 MB image with node_modules
printf 'node_modules\n.git\n' > .dockerignore
podman build -q -f Dockerfile.all -t t2 .          # 8.7 MB
```

Question 3 — `EXPOSE` and the port that does not answer

Answer. No. `EXPOSE` is a **declaration**: it documents that the application listens on 8080 and feeds `podman ps` and `-P`. It creates no redirection from the host. Without `-p 8080:8080`, the port is only reachable from the container network.

Why. Publishing a port is a deployment decision (which host port, which interface), not a property of the image. The image says "I listen on 8080"; the operator decides "I expose it on 18080".

Nuance. `-P` (capital) automatically publishes all `EXPOSE`d ports on random host ports: that is where the declaration becomes useful. And rootless, `-p 80:8080` fails (privileged port); choose ≥ 1024 or tune `net.ipv4.ip_unprivileged_port_start`.

Example.

```
podman run -d --name a my-api:1.0 && curl -m 2 localhost:8080     # failure
podman run -d --name b -p 8080:8080 my-api:1.0 && curl localhost:8080/actuator/health   #
{"status":"UP"}
podman port b                                                      # 8080/tcp -> 0.0.0.0:8080
```

Question 4 — `RUN java` versus `CMD java`

Answer. A launches the API **during the build**: `RUN` executes the command at construction time, the API starts, never ends… and the build stays stuck (or, if the API exits, the image only contains a useless layer). B is correct: `CMD` records the command to launch at `podman run`.

Why. `RUN` serves to prepare the file system (install, compile, copy); `CMD` / `ENTRYPOINT` describe the main process of the future container. Confusing the two is confusing construction and execution.

Nuance. B would be even better with `ENTRYPOINT` + `CMD` (question 5) and a `USER`. And a `RUN java -jar` has a legitimate use: launching a **finite task** at build time, like `java -Djarmode=tools -jar app.jar extract` (lab 05).

Example.

```
podman build -f A -t a .        # STEP 3/3: RUN java -jar … — never completes
podman build -f B -t b . && podman run -d -p 18080:8080 b
```

Question 5 — `ENTRYPOINT` + `CMD` versus `CMD` alone

Answer. A: `podman run img` → `java -jar /app/api.jar --spring.profiles.active=prod`; `podman run img --debug` → `java -jar /app/api.jar --debug` (the `CMD` is replaced, the `ENTRYPOINT` stays). B: `podman run img` → the same full command; `podman run img --debug` → runs **`--debug` on its own**, without `java`: error `executable file not found`. Only B allows `podman run img sh` (the whole `CMD` is replaced by `sh`). With A, `podman run img sh` runs `java -jar api.jar sh`; you need `podman run --entrypoint sh img`.

Why. The arguments of `run` replace the `CMD` and are appended to the `ENTRYPOINT`. A is made for an "application" image, B for a "tool" image.

Nuance. In *exec* form, both A and B put `java` as PID 1. A common company variant: `ENTRYPOINT ["java","-jar","app.jar"]` without `CMD`, and configuration through environment variables — arguments are only for debugging.

Example.

```
timeout 5 podman run --rm api-lab:1.0 --debug | head -1     # Arguments recus : --debug
podman run --rm --entrypoint sh api-lab:1.0 -c 'echo ok'    # ok
```

Question 6 — Ten seconds, `resorting to SIGKILL`, no hooks

Answer. `CMD java -jar /app/api.jar` is a **shell** form: the engine runs `/bin/sh -c "java -jar /app/api.jar"`. On a Debian/Ubuntu base, `/bin/sh` is `dash`, which launches Java as a child and stays PID 1. `podman stop` sends `SIGTERM` to PID 1 — the shell — which does not forward it. Java receives nothing, its *shutdown hooks* do not run; after 10 seconds, Podman announces `resorting to SIGKILL` and kills everything (`137`). Fix: `CMD ["java","-jar","/app/api.jar"]`. On Alpine, `/bin/sh` is `ash` (busybox), which **replaces itself** with the command when it is simple: Java becomes PID 1, receives `SIGTERM`, and the problem is invisible in testing.

Why. A POSIX shell has no obligation to forward signals to its children; `dash` does not. The *exec* form removes the shell, hence the question.

Nuance. Even Alpine does not save a shell `CMD` with `&&`, `|` or a variable: the shell must then stay. The rule "*exec* form always" avoids having to know each shell's behaviour.

Example.

```
podman exec s-deb ps -o pid,args | head -3    # 1 /bin/sh -c java …  2 java …
time podman stop s-deb                        # resorting to SIGKILL, 10 s, code 137
```

Question 7 — `$JAVA_OPTS` not interpreted

Answer. In *exec* form there is **no shell**: `$JAVA_OPTS` is passed to Java as is, a six-character string. Two fixes: (1) explicit shell form with `exec` — `ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]`: cost, a dependency on `sh` and a less readable line; (2) drop the variable — Java itself reads `JAVA_TOOL_OPTIONS` from the environment, so `ENV JAVA_TOOL_OPTIONS="-Xmx512m"` and `ENTRYPOINT ["java","-jar","/app/api.jar"]`: cost, a `Picked up JAVA_TOOL_OPTIONS` message on `stderr` at start-up, and a variable that applies to *all* Java processes in the container.

Why. Variable expansion is a shell service. The *exec* form is an array of arguments passed directly to the `execve` system call.

Nuance. Form (1) without `exec` would recreate the problem of question 6. And for memory specifically, `-XX:MaxRAMPercentage=75` is better than a fixed `-Xmx`: the JVM adapts to the cgroup (lab 10).

Example.

```
podman run --rm -e JAVA_TOOL_OPTIONS="-Xmx256m" api-lab:1.0 --debug 2>&1 | head -1
# Picked up JAVA_TOOL_OPTIONS: -Xmx256m
```

Question 8 — `--build-arg` and the secret

Answer. No. An `ARG`'s value is not in `Config.Env`, but it is recorded in the **history** of every instruction that uses it, and in the build cache. `podman history --no-trunc api:1.0` shows it in the clear (`|1 DB_PASSWORD=Secr3t! /bin/sh -c …`). Anyone with the image has the password.

Why. The history describes how each layer was produced, arguments included — that is what makes the cache possible. An `ARG` is an input of the build, therefore of the cache, therefore of the history.

Nuance. The good practice: the secret has no business at *build* time. If it is indispensable (private Maven repository), `RUN --mount=type=secret,id=settings …` makes it available during one instruction without ever writing it to a layer (lab 08). And a multi-stage build only protects if the secret is used only in a discarded stage (lab 05).

Example.

```
podman history --no-trunc api-lab:arg --format '{{.CreatedBy}}' | grep DB_PASSWORD
# |1 DB_PASSWORD=Secr3t! /bin/sh -c echo "build with $DB_PASSWORD" > /trace.txt
```

Question 9 — Six minutes per Maven build

Answer.

```
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline      # dependency download, cached
COPY src ./src
RUN mvn -q package -DskipTests        # compilation only
```

The cache reuses an instruction if its text **and** its inputs are unchanged. `pom.xml` rarely changes: the `dependency:go-offline` layer (the five minutes of download) stays `Using cache`. Only the compilation is replayed when a `.java` changes. The build will remain slow when `pom.xml` changes — adding or bumping a dependency — since the dependencies layer is then invalidated.

Why. `COPY . /app` places *all* the code before Maven; any commit invalidates the copy, hence everything that follows.

Nuance. `dependency:go-offline` is not perfect (some plugins still download at `package`). A `RUN --mount=type=cache,target=/root/.m2` (lab 05) keeps the local repository between builds, even when `pom.xml` changes. And with Podman, none of these gains depends on a daemon: the cache is in your user storage.

Example.

```
time podman build -t api-lab:2.1 .     # RUN … sleep 5: --> Using cache; 0.7 s
```

Question 10 — Three apt `RUN`s

Answer. (1) **Three layers instead of one**: the apt lists (`/var/lib/apt/lists`, ~40 MB) are written in layer 2; layer 3 only hides them, the image keeps the 40 MB. (2) **Isolated `apt-get update`** gets cached: weeks later, a change to the `install` line will reuse stale indexes (packages not found, old versions). (3) **No `--no-install-recommends`** and `vim` in a production image: dozens of MB of useless packages, as much attack surface. Correct version:

```
RUN apt-get update \
 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*
```

Why. A layer is immutable; clean-up only has an effect in the layer that created the files. And the cache works instruction by instruction: `update` and `install` must go together.

Nuance. On Alpine: `apk add --no-cache curl` does it all in one line. And `vim` in an image is never justified: `podman exec` with a temporary editor, or no editor at all (distroless image, lab 05).

Example.

```
podman history img --format 'table {{.Size}}\t{{.CreatedBy}}'   # the "rm -rf" layer is 0B, the one above 40 MB
```

Question 11 — `Using cache` then everything rebuilt

Answer. The rule: **an invalidated instruction invalidates all those that follow it**, and the cache is read from top to bottom. Day 1: only the last `COPY` changed, everything before it is identical → eight `Using cache`, then rebuild of the last two. Day 2: an instruction inserted in third position changes the text of the Dockerfile from line 3 on → steps 3 to 10 are new, hence rebuilt — even if their content did not move.

Why. The cache key of a step is (parent layer, instruction, inputs). Changing the parent layer changes the key of everything that follows.

Nuance. That is why variable `ENV`, `ARG` and `LABEL` (build number, date) go **at the end** of a Dockerfile, and stable metadata goes early. An `ARG BUILD_DATE` on line 2 invalidates everything, at every build.

Example.

```
podman build -t api-lab:3.1 .     # STEP 3/5: COPY … (replayed) then STEP 4/5: RUN … sleep 5 (replayed too)
```

Question 12 — `ADD` versus `COPY`

Answer. Two behaviours specific to `ADD`: it automatically **unpacks** a local archive (`.tar`, `.tar.gz`, `.tar.xz`) to the destination, and it **downloads** a URL. The official recommendation is `COPY` because these behaviours are implicit: an `ADD file.tar.gz /app/` that unpacks when you wanted to copy the archive, a URL downloaded without verification or cache and with no `rm` possible in the same layer. The only justified case: extracting a **local** archive in one instruction (`ADD rootfs.tar.gz /`).

Why. A Dockerfile must be readable without surprises; `COPY` does one thing. For a URL, `RUN curl … && tar … && rm …` in a single `RUN` is explicit and cleanable.

Nuance. Both accept `--chown` (and `--chmod`), useful before a `USER`. And `COPY --from=` (lab 05) has no `ADD` equivalent.

Example.

```
ADD app.tar.gz /opt/            # /opt/app/… unpacked
COPY app.tar.gz /opt/           # /opt/app.tar.gz as is
```

Question 13 — `USER` too early, and `HUSER 100999`

Answer. `USER` applies to all following instructions, `RUN` included. Placed after `FROM`, it makes `apt-get install` and `mkdir` run as UID 1000, which has no right to write in `/usr`, `/var` or `/`: `Permission denied`. In a well-written Dockerfile, `USER` goes **right before** `ENTRYPOINT` / `CMD`, after installing, creating folders and adjusting their owner (`chown`, `COPY --chown`). Rootless, the container's UID 1000 is mapped onto the host through `/etc/subuid`: the first "extra" UID (1) corresponds to 100000, so 1000 → 100999. `USER` remains useful: (a) it strips the application of root rights *inside* the container (modifying the image files, listening on 80, installing packages); (b) the same image will run under Docker or Kubernetes, where root really is root; (c) security scanners and admission policies reject images without `USER`.

Why. The `user` namespace protects the *host*; `USER` protects the *container* and what it contains. The two layers are complementary.

Nuance. `USER 1000:1000` without creating the user works (Podman even adds an `/etc/passwd` entry on the fly), but some programs want a `HOME` or a name: `RUN adduser -D -u 1000 app` then `USER app` is more robust.

Example.

```
podman top u user,huser          # 1000  100999
podman run --rm --entrypoint sh api-lab:user-ok -c 'touch /app/x'    # Permission denied: good.
```

Question 14 — "A single `RUN`"

Answer. They are right when files created by one instruction are deleted by another (install + clean-up, unpack + delete the archive): separated, the files remain in the image. They are wrong when the grouping mixes stable and volatile: a single `RUN` that downloads the dependencies **and** compiles the code is invalidated at every commit, so re-downloads everything; and a single 300 MB layer is retransferred entirely at every `push`, whereas five layers of which four are stable only transfer the delta.

Why. The number of layers has almost no cost in itself. What matters is **which files live in which layer** (size) and **how often each layer changes** (cache and transfer).

Nuance. Practical rule: one `RUN` per "unit of change" — system installation (stable), dependencies (semi-stable), code (volatile). Multi-stage (lab 05) and the Spring Boot JAR layering apply exactly that logic.

Example.

```
podman history my-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}'   # one layer per unit of change
```